# The Complete Machine Learning Reference Guide

## 1. Standard ML Workflow (7 Steps)

| Step | Name | Key Action | Common Pitfall |
|---|---|---|---|
| **1** | Define problem | Regression (number), classification (category), clustering (group), or recommendation? | Solving wrong metric (e.g., accuracy for rare disease) |
| **2** | Collect data | Ensure labels exist for supervised learning | Leaking future information into training |
| **3** | Clean & preprocess | Handle missing values, encode categories, scale features | Applying scaling *before* train/test split |
| **4** | Split data | Train (60%), validation (20%), test (20%) | Testing on the same data you tuned on |
| **5** | Establish baseline | Simple model (mean, median, or linear regression) | Starting with deep learning immediately |
| **6** | Train & tune | Cross-validation + hyperparameter search | Overfitting to validation set |
| **7** | Evaluate & deploy | Compare to baseline on test set | Ignoring inference latency / memory constraints |

## 2. Model Categories – When to use each.

| Model Family | Best used for | Key benefits | Critical limits |
|---|---|---|---|
| **Linear Models** (Ridge, Lasso, Logistic) | Baseline, high-dimensional sparse data, interpretability required | Very fast, coefficients = feature importance | Assumes linear relationship, fails on XOR/nonlinear patterns |
| **Tree-Based** (Random Forest, XGBoost, LightGBM) | Tabular data (sales, finance, healthcare), nonlinear patterns, missing values | No scaling needed, handles mixed types, built-in feature importance | Poor performance on images/text/graphs, can overfit |
| **Support Vector Machines** (SVM / SVR) | Small-to-medium tabular (<10k rows), high-dim but sparse | Kernel trick captures complex boundaries | $O(n^2)$ to $O(n^3)$ time, memory heavy, needs scaling |
| **K-Nearest Neighbors** (KNN) | Small datasets, low-dimension (<20 features), simple recommendations | No training phase, easy to explain | Slow inference O(n), curse of dimensionality |
| **Neural Networks / Deep Learning** | Images (CNN), text/sequences (Transformers), audio, large tabular (>100k rows) | State-of-the-art for unstructured data | Needs large data + GPU, black-box, many hyperparameters |
| **Clustering** (K-Means, DBSCAN, Hierarchical) | Unsupervised grouping, customer segmentation, anomaly detection | No labels needed, reveals hidden structure | Hard to evaluate, K-Means assumes spherical clusters |
| **Dimensionality Reduction** (PCA, t-SNE, UMAP) | Visualization, feature compression, denoising | Reduces overfitting, speeds up other models | PCA is linear, t-SNE distorts global distances |

## 3. What to use for What Exactly

| Problem type | Start with | Then try | Avoid |
|---|---|---|---|
| House price prediction (regression) | Linear Regression (baseline) | XGBoost, Random Forest | Deep learning (unless >1M rows) |
| Spam / fraud detection (binary classification) | Logistic Regression | XGBoost, LightGBM | SVM (too slow for large data) |
| Multi-class classification (iris, product category) | Random Forest | XGBoost, KNN | Neural net (overkill for small data) |
| Image classification | Pretrained ResNet (fine-tune) | EfficientNet, ViT | KNN, linear models, or training from scratch |
| Sentiment analysis / text classification | Naive Bayes + TF-IDF | DistilBERT, Logistic Regression with TF-IDF | RNNs (use Transformers instead) |
| Customer segmentation (unsupervised) | K-Means | DBSCAN, Gaussian Mixture | Hierarchical clustering (if >5k rows) |
| Anomaly detection (fraud, manufacturing defects) | Isolation Forest | DBSCAN, Autoencoder | K-Means (forces all points into clusters) |
| Recommendation system (user-item) | Collaborative filtering (SVD) | LightGCN, Two-tower neural net | KNN (scales poorly beyond small datasets) |
| Time series forecasting (sales, demand) | Seasonal Naive or Exponential Smoothing | XGBoost with lag features, Prophet | LSTM (often worse than lagged tree models) |
| Extremely large tabular (>1M rows) | LightGBM or CatBoost | Random Forest with subsampling | SVM, KNN, standard deep nets (without GPU) |

## 4. Model Benefits and Limits

| Model | Benefits | Limits | Need scaling? | Handles missing values? |
|---|---|---|---|---|
| Linear Regression | Interpretable, extremely fast | Only linear relationships, outlier-sensitive | Yes | No |
| Logistic Regression | Probability outputs, fast training | Linear decision boundary | Yes | No |
| Decision Tree | No scaling, human-readable rules | Overfits easily, unstable | No | Yes (most implementations) |
| Random Forest | Robust, no overfit with enough trees | Slower inference, large memory footprint | No | Yes |
| XGBoost | Top accuracy for tabular, handles sparse data | Many hyperparameters, can overfit if not tuned | No | Yes (treats missing as zero) |
| SVM (RBF kernel) | Works well in high dimensions | Very slow training/prediction, memory heavy | Yes (critical) | No |
| KNN | Simple, no training phase | Slow inference, curse of dimensionality | Yes (essential) | No |
| Neural Network | SOTA for unstructured data, highly flexible | Data hungry, black box, needs GPU/tuning | Yes | Only if imputed |

| | | | | |
|---|---|---|---|---|
| K-Means | Fast, easy to implement | Assumes spherical clusters, requires specifying K | Yes | No |
| PCA | Speeds up other models, removes noise | Linear only, loses feature interpretability | Yes | No |

# 5. Practical adoption rules

✓ Rule 1: Always start simple – linear or naive baseline first. You need a lower bound.

✓ Rule 2: Tabular data → XGBoost/LightGBM (wins ~90% of Kaggle tabular comps).

✓ Rule 3: Images/text → pretrained deep learning. Don't train from scratch unless >100k labeled examples.

✓ Rule 4: Small data (<1k rows) → linear models or KNN + feature engineering. Avoid deep learning.

✓ Rule 5: Missing values → tree models handle them natively; linear models need imputation.

✓ Rule 6: Interpretability needed → linear models, small decision trees, or SHAP on XGBoost.

✓ Rule 7: Inference speed critical (real-time) → logistic regression, small tree ensemble, or quantized neural net.

✓ Rule 8: Unsupervised → start with K-Means (if equal cluster sizes) or DBSCAN (if density-based).

✓ Rule 9: Time series → always start with seasonal naive or exponential smoothing before ML.

✓ Rule 10: When in doubt → validate using 5-fold cross- validation + compare test set to baseline.

# 6. Quick Reference: One line Summaries

**Linear models:** Fast, interpretable, but only straight-line patterns. **Random**
**Forest:** Reliable default for tabular, hard to mess up. **XGBoost/LightGBM:**
Highest accuracy for tabular – your go-to after baseline.
**Neural networks:** Only for unstructured data (images, text, audio) or massive tabular.
**KNN:** Only for tiny, low-dimensional datasets where you need zero training.
**SVM:** Legacy – rarely needed today (XGBoost or MLP usually better).
**K-Means:** Fast segmentation when clusters are round and equal size.
**DBSCAN:** Clustering when you don't know number of clusters and have irregular shapes.
**PCA:** Use before linear models or KNN to reduce dimensions and speed up.